

John the Ripper

Tools Used

- John the Ripper: A powerful password-cracking tool.
- Wordlists: Common password files for dictionary attacks.
- Hashcat: Can be introduced for comparison with John the Ripper, though focus will remain on John the Ripper.

Lab Steps

Step 1: Installing John the Ripper

1. Ensuring John the Ripper is Installed:

- In Kali Linux, John the Ripper comes pre-installed, but if not:

```
bash
```

```
sudo apt install john
```

2. Verify the installation by checking the version:

```
bash
```

```
john --version
```

Exercise 1:

- What version of John the Ripper are you using?

John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c

Step 2: Understanding Password Hashes

1. Identify Hash Type:

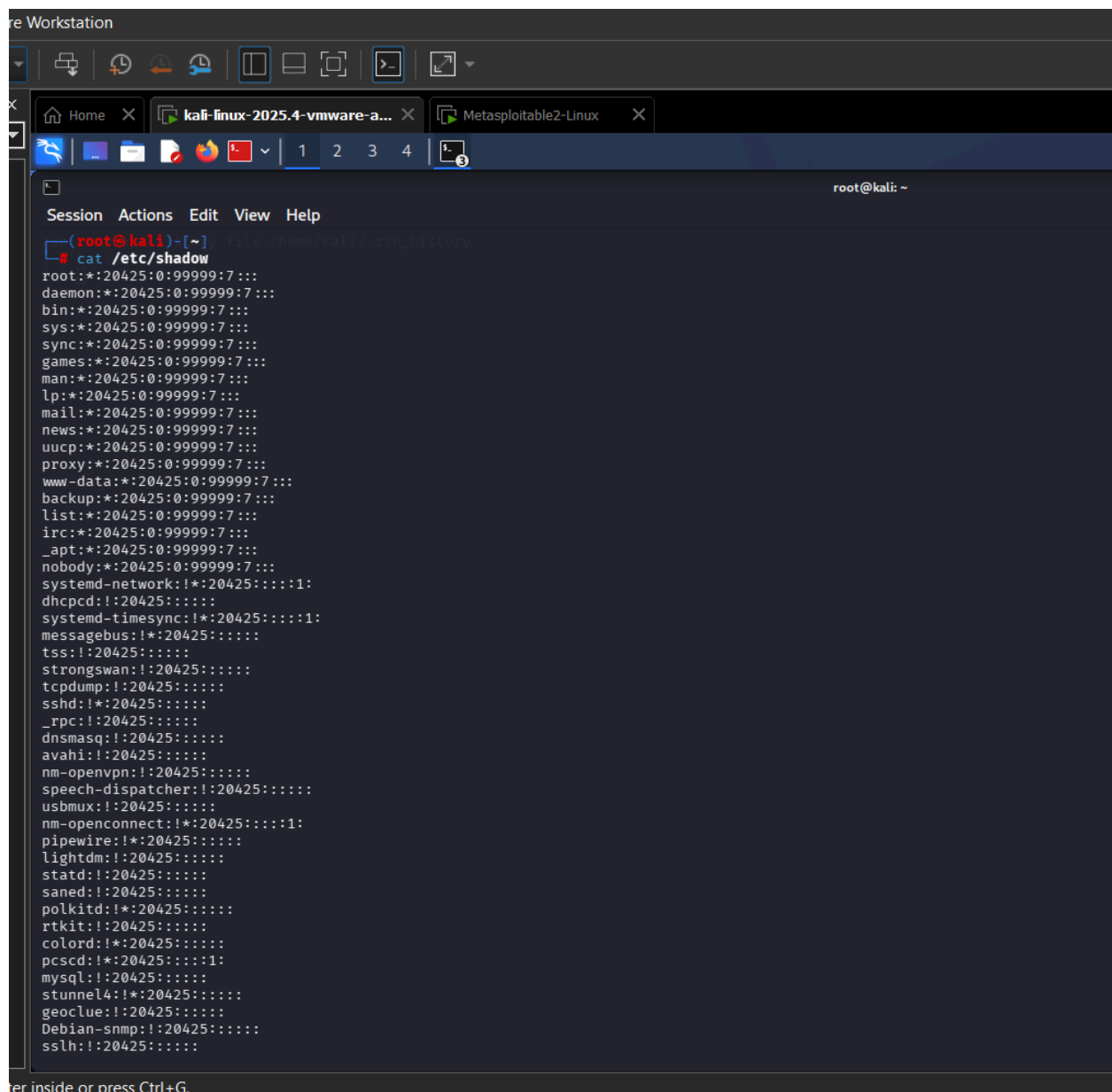
- Before cracking passwords, identify the hash type. For example,

you can check hash examples here:

bash

cat /etc/shadow

- Observed the hashed passwords in the `/etc/shadow` file.



The screenshot shows a Kali Linux terminal window with the command `cat /etc/shadow` executed. The output displays the system's password file, listing various users and their hashed passwords. The users listed include root, daemon, bin, sys, sync, games, man, lp, mail, news, uucp, proxy, www-data, backup, list, irc, _apt, nobody, systemd-network, dhcpcd, systemd-timesync, messagebus, tss, strongswan, tcpdump, sshd, _rpc, dnsmasq, avahi, nm-openvpn, speech-dispatcher, usbmux, nm-openconnect, pipewire, lightdm, statd, saned, polkitd, rtkit, colord, pcscd, mysql, stunnel4, geoclue, Debian-snmpp, and sslh. Each entry follows the format `username:hash:expiration:warning:inactive:expire:login`.

```
root@kali: ~  
Session Actions Edit View Help  
(root@kali)-[~]  
# cat /etc/shadow  
root:*:20425:0:99999:7:::  
daemon:*:20425:0:99999:7:::  
bin:*:20425:0:99999:7:::  
sys:*:20425:0:99999:7:::  
sync:*:20425:0:99999:7:::  
games:*:20425:0:99999:7:::  
man:*:20425:0:99999:7:::  
lp:*:20425:0:99999:7:::  
mail:*:20425:0:99999:7:::  
news:*:20425:0:99999:7:::  
uucp:*:20425:0:99999:7:::  
proxy:*:20425:0:99999:7:::  
www-data:*:20425:0:99999:7:::  
backup:*:20425:0:99999:7:::  
list:*:20425:0:99999:7:::  
irc:*:20425:0:99999:7:::  
_apt:*:20425:0:99999:7:::  
nobody:*:20425:0:99999:7:::  
systemd-network:!:*:20425::::1:  
dhcpcd:!:20425::::1:  
systemd-timesync:!:*:20425::::1:  
messagebus:!:*:20425::::1:  
tss:!:20425::::1:  
strongswan:!:20425::::1:  
tcpdump:!:20425::::1:  
sshd:!:*:20425::::1:  
_rpc:!:20425::::1:  
dnsmasq:!:20425::::1:  
avahi:!:20425::::1:  
nm-openvpn:!:20425::::1:  
speech-dispatcher:!:20425::::1:  
usbmux:!:20425::::1:  
nm-openconnect:!:*:20425::::1:  
pipewire:!:*:20425::::1:  
lightdm:!:20425::::1:  
statd:!:20425::::1:  
saned:!:20425::::1:  
polkitd:!:*:20425::::1:  
rtkit:!:20425::::1:  
colord:!:*:20425::::1:  
pcscd:!:*:20425::::1:  
mysql:!:20425::::1:  
stunnel4:!:*:20425::::1:  
geoclue:!:20425::::1:  
Debian-snmpp:!:20425::::1:  
sslh:!:20425::::1:
```

Enter inside or press Ctrl+G.

2. Popular Hash Formats:

- MD5, SHA-1, and DES are common formats that John can handle.

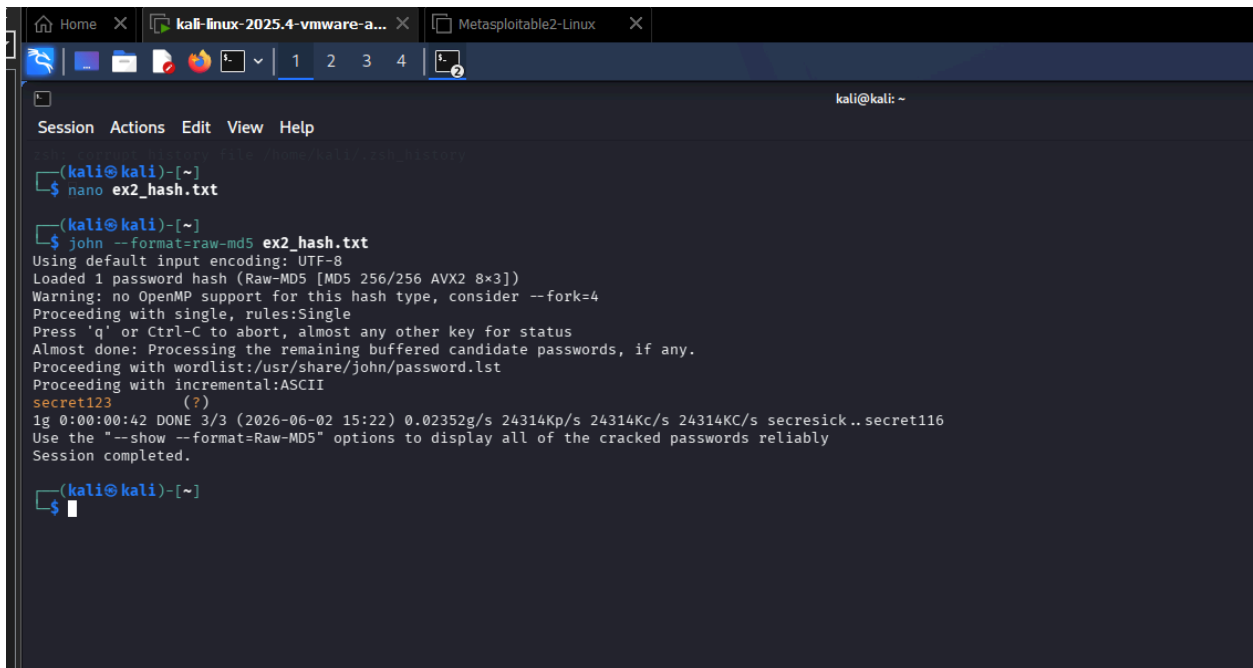
Exercise 2:

- Using John the Ripper, how do you identify the type of a given hash?
- Run the following command on sample hashes:

"5d7845ac6ee7cffffafc5fe5f35cf666d"

bash

john --format=raw-md5 <hashfile>



```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~  
$ nano ex2_hash.txt  
[kali@kali]~  
$ john --format=raw-md5 ex2_hash.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Proceeding with single, rules:Single  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Almost done: Processing the remaining buffered candidate passwords, if any.  
Proceeding with wordlist:/usr/share/john/password.lst  
Proceeding with incremental:ASCII  
secret123 (?)  
1g 0:00:00:42 DONE 3/3 (2026-06-02 15:22) 0.02352g/s 24314Kp/s 24314Kc/s 24314KC/s secre sick..secret116  
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably  
Session completed.  
[kali@kali]~  
$
```

Created hashes by using the method :

bash

echo -n "secret123" | md5sum

Save it into a new file

nano ex2_hash.txt

Paste into nano file hashes generated:

"5d7845ac6ee7cfffafc5fe5f35cf666d"

Ctrl + O to Save

Enter

X to exit.

Run John to identify format

john --format=raw-md5 ex2_hash.txt

Successfully identified hash type:

Hash type identified: MD5 (raw-md5)

Tool used: John the Ripper

Method: Specified format using --format=raw-md5

Step 3: Cracking Passwords with Wordlists

1. Using Default Wordlist:

- Use the default wordlist (`rockyou.txt`) to attempt cracking a simple MD5

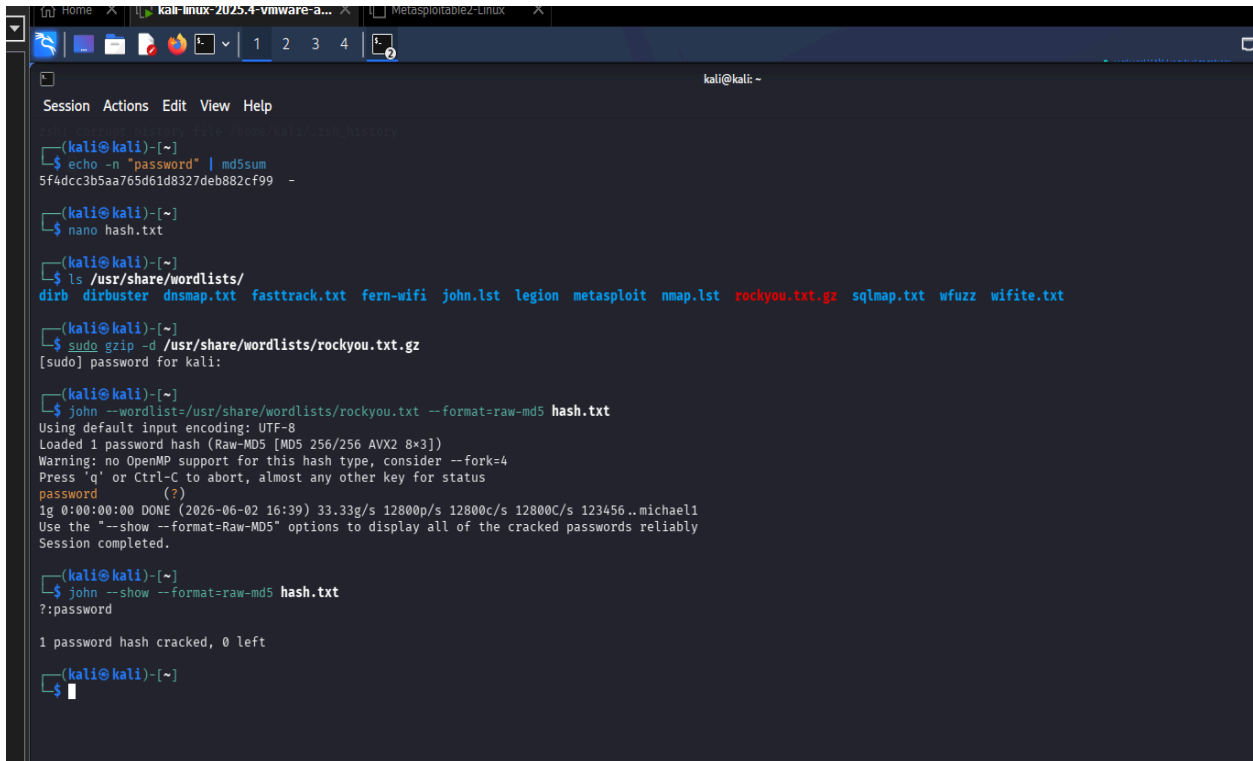
hash:

bash

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Exercise 3:

- Download a sample hash and crack it using the wordlist. What was the password? Was it successful?



```
(kali@kali)-[~]
└─$ echo -n "password" | md5sum
5f4dcc3b5aa765d61d8327deb882cf99 -

(kali@kali)-[~]
└─$ nano hash.txt

(kali@kali)-[~]
└─$ ls /usr/share/wordlists/
dirb  dirbuster  dnsmap.txt  fasttrack.txt  fern-wifi  john.lst  legion  metasploit  nmap.lst  rockyou.txt.gz  sqlmap.txt  wfuzz  wifite.txt

(kali@kali)-[~]
└─$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
[sudo] password for kali:

(kali@kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5 hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
password
(?)
1g 0:00:00:00 DONE (2026-06-02 16:39) 33.33g/s 12800p/s 12800c/s 12800C/s 123456..michael1
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~]
└─$ john --show --format=raw-md5 hash.txt
?:password

1 password hash cracked, 0 left

(kali@kali)-[~]
└─$
```

Successfully created a sample hash and cracked it using wordlist.

Password: password

Successful: Yes

2. Adding Custom Wordlists:

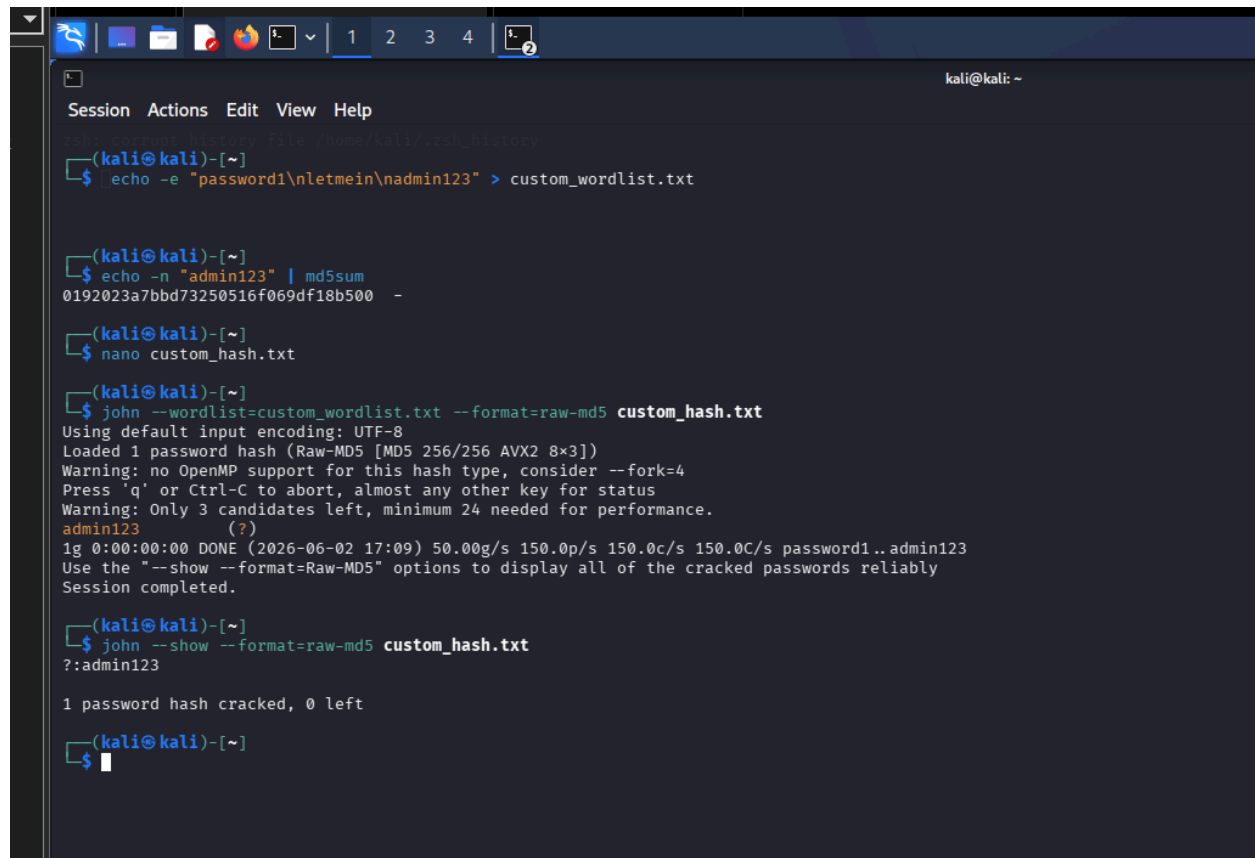
- Create a custom wordlist file with possible passwords:

bash

```
echo -e "password1\nletmein\nadmin123" > custom_wordlist.txt
```

Exercise 4:

- Run John with your custom wordlist on a given hash. Was your list successful in cracking the hash? _____



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ echo -e "password1\nletmein\nadmin123" > custom_wordlist.txt  
  
(kali@kali)-[~]  
$ echo -n "admin123" | md5sum  
0192023a7bbd73250516f069df18b500 -  
  
(kali@kali)-[~]  
$ nano custom_hash.txt  
  
(kali@kali)-[~]  
$ john --wordlist=custom_wordlist.txt --format=raw-md5 custom_hash.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Warning: Only 3 candidates left, minimum 24 needed for performance.  
admin123 (??)  
1g 0:00:00:00 DONE (2026-06-02 17:09) 50.00g/s 150.0p/s 150.0c/s 150.0C/s password1..admin123  
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably  
Session completed.  
  
(kali@kali)-[~]  
$ john --show --format=raw-md5 custom_hash.txt  
?:admin123  
  
1 password hash cracked, 0 left  
  
(kali@kali)-[~]  
$
```

Successfully created a sample hash and cracked it using a custom wordlist.

Password: admin123

Successful: Yes

Step 4: Brute Force Mode

1. Understanding Brute Force:

- If the wordlist attack fails, you can switch to brute force mode:

```
bash
```

```
john --incremental hash.txt
```

2. Time and Complexity:

- Brute force takes longer but tries every possible combination.

Exercise 5:

- Perform a brute force attack on a hash. How long did the attack take, and was it successful?

The screenshot shows a Kali Linux terminal window with the following commands and output:

```
Session Actions Edit View Help
$ cat /dev/null | md5sum
(kali@kali)-[~]
$ echo -n "abc" | md5sum
900150983cd24fb0d6963f7d28e17f72 -
(kali@kali)-[~]
$ nano brute.txt
(kali@kali)-[~]
$ john --incremental --format=raw-md5 brute.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
abc (?)
1g 0:00:00:00 DONE (2026-06-02 17:29) 3.225g/s 232877p/s 232877c/s 232877C/s lesua..lank
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
(kali@kali)-[~]
$ john --show --format=raw-md5 brute.txt
?:abc
1 password hash cracked, 0 left
(kali@kali)-[~]
$
```

Successfully created a sample NTLM hash and cracked it using a brute force attack.

Password: abc

How long : 1 minutes

Step 5: Cracking Windows Password Hashes

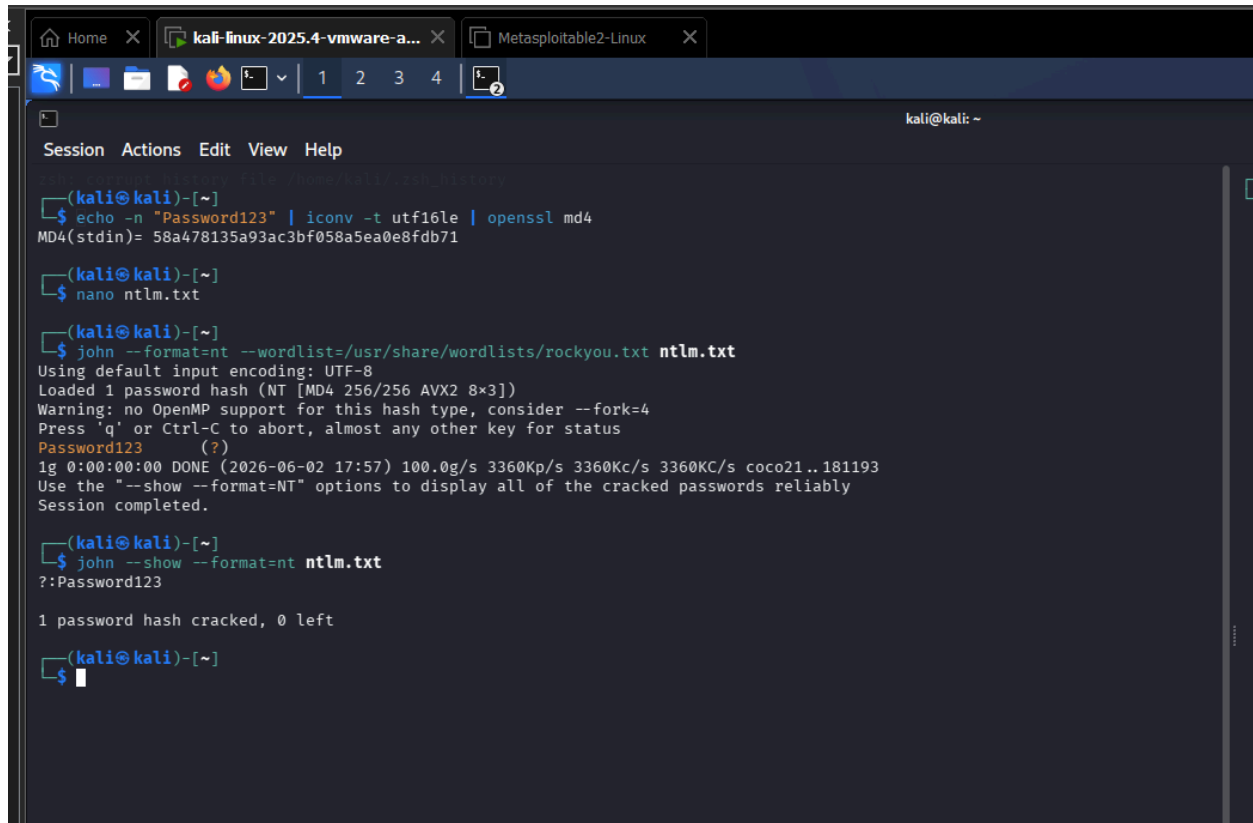
1. Dumping Windows Hashes:

- Extract password hashes from a Windows machine using tools like `pwdump` or `hashdump`.
- Example hash (LM or NTLM):

```
john --format=nt hashfile
```

Exercise 6:

- Attempt cracking NTLM hashes using the `rockyou.txt` wordlist.
- Were you successful? How complex was the password?



```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~  
$ echo -n "Password123" | iconv -t utf16le | openssl md4  
MD4(stdin)= 58a478135a93ac3bf058a5ea0e8fdb71  
[kali@kali]~  
$ nano ntlm.txt  
[kali@kali]~  
$ john --format=nt --wordlist=/usr/share/wordlists/rockyou.txt ntlm.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Password123 (?)  
1g 0:00:00:00 DONE (2026-06-02 17:57) 100.0g/s 3360Kp/s 3360Kc/s 3360KC/s coco21..181193  
Use the "--show --format=NT" options to display all of the cracked passwords reliably  
Session completed.  
[kali@kali]~  
$ john --show --format=nt ntlm.txt  
?:Password123  
  
1 password hash cracked, 0 left  
[kali@kali]~  
$
```

Successfully created a sample hash and cracked it using the `rockyou.txt` wordlist.

Password complexity:Medium

Successful: Yes

Step 6: Advanced Cracking with Rules

1. Adding Rules for More Power:

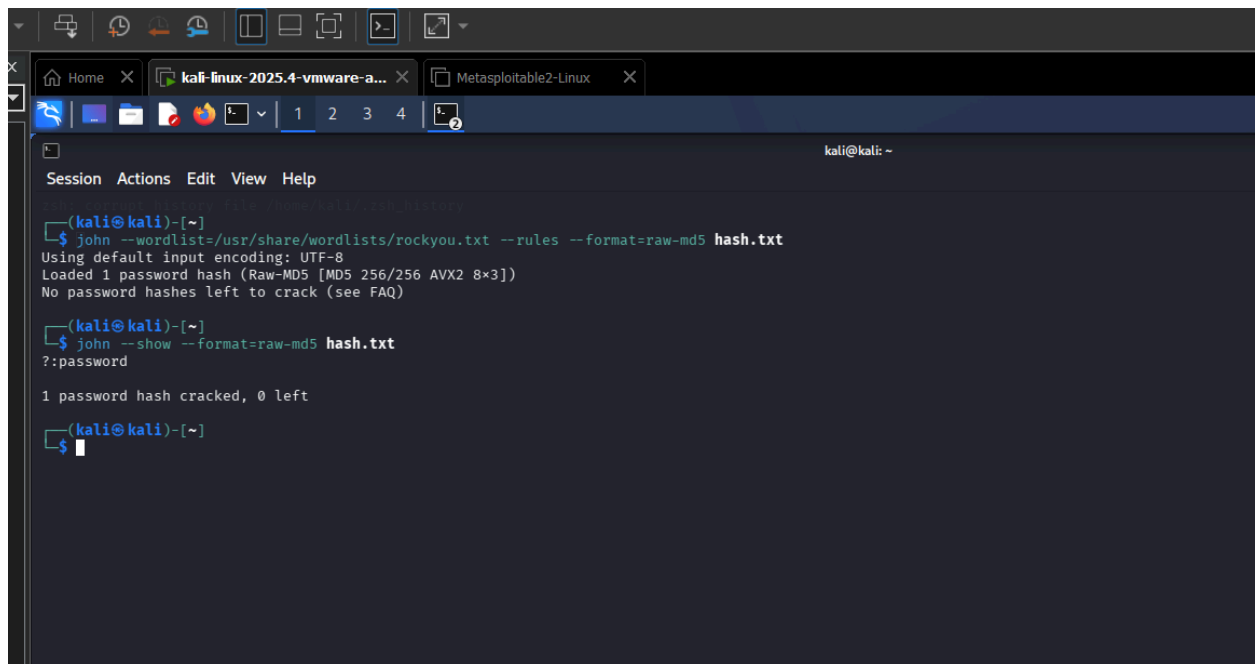
- John the Ripper allows you to create rules that modify the wordlist:

bash

```
john --wordlist=rockyou.txt --rules --format=raw-md5 hash.txt
```

Exercise 7:

- Use rules with a wordlist attack to crack a complex password. What was the result?

A screenshot of a terminal window on a Kali Linux system. The terminal shows the execution of John the Ripper with a wordlist and rules. The output indicates that one password hash was loaded and successfully cracked. The terminal window has a dark background with light-colored text. The top of the window shows a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below the menu bar, the terminal prompt is '(kali@kali)-[~]'. The user enters the command 'john --wordlist=/usr/share/wordlists/rockyou.txt --rules --format=raw-md5 hash.txt'. The output shows 'Using default input encoding: UTF-8', 'Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])', and 'No password hashes left to crack (see FAQ)'. The user then enters 'john --show --format=raw-md5 hash.txt'. The output shows '?:password' and '1 password hash cracked, 0 left'. The terminal window is titled 'kali-linux-2025.4-vmware-a...' and 'Metasploitable2-Linux'.

Successfully cracked a complex password .

Password:password

Successful: Yes

Analysis and Conclusion

- Wordlist attacks had a high success rate against common passwords and were the fastest method.
- Custom wordlist attacks were successful when the correct password was included in the custom list.
- Brute force attacks can crack passwords not found in wordlists, but they take significantly more time and computing resources.
- Rule-based attacks improved success rates by generating variations of existing words in a wordlist.

Importance of Strong Passwords

- Longer passwords are harder to crack because they increase the number of possible combinations.
- Complex passwords that use uppercase letters, lowercase letters, numbers, and special characters are more resistant to wordlist and brute-force attacks.
- Salts make password hashes unique, preventing attackers from using precomputed hash tables (rainbow tables) and making cracking more difficult.
- Using strong, unique passwords and modern hashing algorithms with salts greatly improves account security.

